NeuroLog AI, explained

1 / 5 WHAT IT DOES AND HOW THE PIECES CONNECT

The purpose: collect messages from a running application, show them in a dashboard, group similar messages, and help investigate unusual activity. A **log** is a short written record of an event: for example, "Task created task_id=12". Logs help explain what happened, when it happened, and which service reported it.

The running architecture

```
Python task app :8000
  -> Python logging -> SQLite outbox -> background sender
  -> Flask API :5001 -> SQLite log database
  -> React dashboard :5173 -> your browser
                        -> ML groups / incident assistant
```

Frontend means the screens you see. **Backend** means the server that validates requests and performs work. An **API** is the agreed set of addresses and input formats through which they communicate. **HTTP** carries requests; **JSON** represents data as fields and values. A **port** selects a service on a machine. **127.0.0.1**, or localhost, means your own computer.

Follow one task, all the way through

1. In the Python app, you submit a task title. The browser sends POST /tasks. Flask checks that the title contains 1-200 characters, inserts it into tasks.db, and returns status 201, meaning created.

2. Python logging records the task event and the HTTP method, path, response status, and duration in milliseconds. NeuroLogHandler converts the Python severity into a syslog number and saves a JSON payload in the outbox.

3. A background thread sends that payload to /api/ingest with an API key. The backend verifies the key, adds its own UTC receipt timestamp and severity label, and inserts the log into neurolog-live.db.

4. The dashboard periodically requests recent records. React updates the counters, chart, and table. Completing the task uses PATCH /tasks/id with done=true and creates another event. No manual file import is needed.

Which folders matter?

live_app/: the task app, durable sender, synthetic generator, and its HTML screen. **LogIntel_engine/api.py:** current API, storage, clustering, and AI-provider calls. **analysis.py:** local incident rules. **logintel_ui/src/:** React screens, shared components, API helper, themes, and CSS.

tests/: automated checks. **start-preview.ps1:** local launcher. **render_app.py, render_wsgi.py, Dockerfile, render.yaml:** hosted packaging. **docs/:** guides and design notes. **data/:** runtime databases, key file, process IDs, and service output. **output/pdf/:** delivered documents.

Important distinction: the normal launcher runs three services and uses SQLite. The Express server is an optional forwarding gateway. MongoDB helpers and older scripts remain in the repository but are not part of this normal flow. This guide describes the inspected code as of 6 October 2026; five pages cover the main mechanisms and useful small details, not every source line.

Legacy log_courier.py tails new JSON lines into MongoDB; te.py runs keyword rules and health counters on those collections. server/config/db.js connects Mongoose; server/utils/logger.js writes Winston JSON logs. server/server.js now forwards /api/log to Flask. clean_tfidf_vectors.csv, screenshots, and older UI files are not loaded by the current clustering route or router. Sources: the files named in this guide. MIT attribution remains in LICENSE/README.

The backend and its data

Flask and SQLite in plain language

Flask maps an address such as /api/ingest to a Python function. create_app() builds an application; tests can pass temporary settings without starting a network server. **SQLite** keeps structured tables in a file. SQL INSERT adds a row, SELECT reads rows, UPDATE changes them, and DELETE removes them. commit() saves the transaction. The code uses ? placeholders so input is handled as data rather than SQL code.

The API opens one SQLite connection per request context and closes it afterward. A 10-second connection timeout allows waiting for a locked database. The logs_owner_time index is a lookup shortcut on workspace owner and timestamp.

The stored fields

logs: _id is the record number; uid identifies its workspace; timestamp is API receipt time; source names the reporting service; severity_level is its numeric importance; severity_label is readable text; message holds the event; ml_anomaly is 0 or 1; cluster_id is initially NULL, then a group number or -1.

keys: id, owner, app name, SHA-256 digest, creation time, and last six key characters. A digest is a one-way fingerprint used for matching. The raw key is returned at creation; listings show only metadata. **settings:** per-owner retention. **tasks.db:** task id, title, done (0/1). **task-log-outbox.db:** queued id and JSON payload. These databases serve different jobs.

API route reference: /api is the common prefix

GET /health: checks database access and reports mode. **POST /generate-key:** accepts app_name and returns a new key. **GET /get-keys:** lists key metadata. **POST /ingest:** accepts a log with x-api-key. **GET /recent-logs?limit=N:** newest records, default 100, allowed 1-1000.

POST /run-ml: recomputes clustering. **POST /chat:** accepts message and returns an investigation. **GET or POST /settings:** reads or sets retention to 3, 14, or 90 days. **DELETE /logs:** removes workspace logs. The current Settings screen does not expose every backend operation, including retention, key management, and deletion.

```
POST http://127.0.0.1:5001/api/ingest
Content-Type: application/json
x-api-key: YOUR_CREATED_KEY

{"source":"my-service", "severity_level":3,
 "message":"Database connection timeout"}
```

Validation: message is nonempty, up to 10,000 characters; source up to 120, default external-app; severity must be an integer 0-7, default 6. The API body limit is 64 KB. Invalid input returns 400; a missing/invalid ingest key returns 401; unexpected failures return a generic 500 without exposing internal details.

Why logs survive an API outage

The sender queues before sending, handles delivery on a daemon thread, retries at 1, 2, 4 seconds and so on up to 30, and removes an event only after success. A restart can resend queued events. The queue holds at most 10,000 entries; newer entries are dropped when full. It is not exactly-once delivery: a lost response or crash before queue deletion can cause a duplicate.

Local sending can create and cache a key in task-service.key; an automatically obtained invalid key is discarded for regeneration. A remote endpoint requires a configured ingest key. Retention defaults to 14 days: old records are deleted when recent records are read, rather than by a separate scheduled cleanup job.

How the screens work

The frontend building blocks

React 19 builds the interface from reusable components. **JSX** is JavaScript syntax that looks like HTML. **State** is the screen's changing memory: logs, filters, selected records, and loading status. useState holds it; useEffect handles fetching and timers; cleanup stops timers when a screen closes. useRef points to an element, used here for chat scrolling.

Vite runs the local development server and compiles production files into dist/. **React Router** chooses the component for a URL. App.jsx lazy-loads pages, so a page module loads when needed; Suspense shows a loading message. Unknown routes redirect to /dashboard.

Axios sends API requests through src/api.js, using VITE_API_URL or /api, with a 35-second timeout. Its error interceptor dispatches a shared neurolog-error event; App.jsx displays the banner. In local development, vite.config.js forwards /api to port 5001. This proxy lets the browser use one frontend address. CORS is the backend's rule for which browser origins may call it directly.

What each page actually calculates

Overview (/dashboard): requests the latest 300 logs every **3 seconds**. It counts errors with severity <=3, stored ML outliers, and distinct sources. Recharts draws minute buckets for routine versus error records and shows the last 20 buckets. Cards describe this recent window, not lifetime totals. "API connected" means the latest request succeeded, not that every source is healthy.

Log workspace (/explorer): also requests up to 300 every 3 seconds. Search checks message and source; filters select severity, source, and actual/generated origin. These are browser-side filters over fetched records, not a full database search. Selecting a row opens details. Pause stops this screen's refresh; the backend still receives logs.

Anomaly analysis (/analysis): fetches up to 300 every 5 seconds and displays records whose cluster_id is set, group counts excluding -1, and flagged outliers. It reads saved results; visiting this page does not train or rerun analysis. Run analysis explicitly calls /run-ml.

AI investigation (/assistant): refreshes recent evidence every 5 seconds, shows up to 12 error/critical records for the selected source, and sends a question to /chat. The reply displays as Markdown. The server returns whether analysis came from local rules or Groq. Clearing the conversation clears the notebook, not stored logs.

Settings (/configuration): offers dark, light, or system appearance, an analysis button, AI configuration guidance, and a link to the Python application. Sidebar and LogBadge are shared components; index.css and App.css control colors, spacing, layout, and responsive behavior. Lucide provides icons.

What the browser remembers

localStorage retains the theme on this browser/device across sessions. ThemeContext shares it throughout the component tree and sets the HTML data-theme attribute; system mode follows the operating system's color preference. **sessionStorage** retains the latest 50 chat messages for the tab session. Neither is the backend log database, and chat history is not supplied as full conversation history to /chat.

Small details that prevent misunderstandings

The interface uses polling: it repeatedly asks for fresh data rather than using WebSockets to push events. Dates are stored as UTC strings and rendered with the browser's local time. Buttons disable during an analysis or reply to reduce duplicate actions. A skip link, labeled controls, and status/alert regions support keyboard and assistive access. The README mentions a 2-second dashboard interval; the inspected component uses 3 seconds.

The ML and AI, demystified

First: severity is supplied, not predicted

The syslog scale is **0 emergency, 1 alert, 2 critical, 3 error, 4 warning, 5 notice, 6 info, 7 debug**. Smaller numbers mean more urgent. The Python handler maps CRITICAL to 2, ERROR to 3, WARNING to 4, INFO to 6, and lower levels to 7. The backend creates the text label from that number. A serious severity and an unusual message are different signals.

TF-IDF turns text into comparable numbers

Tokenization splits a message into usable words. **TF** measures how much a word appears in a message. **IDF** gives less weight to words common across many messages and more to distinctive words. Together, **TF-IDF** produces a vector: a row of word weights. A sparse matrix stores mostly nonzero weights to save space.

The implementation uses TfidfVectorizer(stop_words="english", max_features=5000). Common English words are removed; at most 5,000 word features are used. It fits vocabulary from the current batch every run. This is text similarity, not a pretrained language model or semantic understanding of the service.

DBSCAN finds dense groups and lonely points

DBSCAN groups vectors near each other. eps=0.5 is the maximum neighborhood distance; min_samples=2 requires at least two points, counting the point itself, for a dense neighborhood. The code uses the library's default Euclidean distance. Similar recurring messages may form a group; an isolated point gets cluster_id=-1 and ml_anomaly=1.

Run analysis uses the **latest 100 logs**, although pages display 300. At least five logs are required. A batch with no usable words returns a message without clustering. Results are saved on the analyzed records; other records keep earlier results. Cluster numbers have meaning only within a run, so a displayed set can mix old and new assignments.

Example: several "Task created" messages may cluster together; a rare "Database connection timeout" may stand alone. But a rare successful event can also be an outlier, and a frequently repeated failure can form a normal-looking group. Changing numbers, vocabulary, or batch composition can change the result. There is no measured accuracy score or trained failure classifier here.

Local incident analysis: transparent rules

Without GROQ_API_KEY, analysis.py inspects up to 300 recent records. It considers warnings/errors and stored outliers, matches wording to database, memory, upstream, rate-limit, or authentication rules, then groups by category, source, and generated status. It provides counts, up to three evidence records per group, and investigation advice. Up to eight groups appear in the reply.

The syntax **source:task-service** filters local evidence deterministically. Other question wording does not enable general conversation; asking about clusters adds a reminder about saved ML results. Rule ordering matters: the first matching category wins. A category is a clue for investigation, not a verified explanation.

Optional Groq language-model mode

With a configured key, /chat sends a question and recent-log JSON to Groq; each log message is truncated to 800 characters. The configured default model is llama-3.3-70b-versatile, temperature 0.2, response limit 1,024 tokens, timeout 30 seconds, and one retry. The system prompt asks for evidence IDs, uncertainty, and separation of generated examples. Provider failure returns a labeled local fallback. This mode sends the selected log context to an external provider.

Synthetic generator: opt-in [GENERATED] messages simulate successful requests, rate limiting, database/upstream errors, and memory failures. Interval is 0.2-10 seconds and limit 1-1,000. A lock prevents simultaneous runs; Stop signals its thread. Generated messages enter the same ingestion and ML pipeline; local incident groups separate them. Neither AI mode executes fixes.

Run, verify, and understand limits

5 / 5 STARTUP, HOSTING, TROUBLESHOOTING, AND PRACTICE

Local startup and useful commands

The README specifies Python 3.12 and Node.js. A virtual environment (.venv) isolates Python dependencies; requirements-lock.txt supplies locked versions. npm ci installs the frontend from its lockfile. The launcher installs missing environments, starts hidden services, records process IDs/output in data/, and checks health. It forces local mode for its services.

```
powershell -ExecutionPolicy Bypass -File .\start-preview.ps1
# Open: http://127.0.0.1:8000
# Open: http://127.0.0.1:5173/dashboard

.venv/Scripts/python.exe -m pytest -q
# From logintel_ui/:
npm run lint
npm run build
```

pytest exercises behavior; **lint** finds JavaScript rule violations; **build** checks production compilation. Existing tests cover ingestion validation, clustering edge cases, retention/persistence, authentication isolation, AI fallback, task events, generator limits, queue recovery, and hosted routes. They are useful evidence but do not measure ML accuracy or prove production security. This guide inspected the test code; it does not claim a fresh test run.

Configuration and access

The API loads .env from the project root. NEUROLOG_DATABASE selects log storage; NEUROLOG_LOCAL_MODE defaults true; CORS_ORIGINS lists allowed origins; GROQ_API_KEY/GROQ_MODEL control the provider. The sender uses NEUROLOG_API_URL and NEUROLOG_INGEST_KEY. Frontend VITE_API_URL and VITE_SOURCE_URL are build-time settings, so changing a production value requires a rebuild.

Local mode treats viewers as local-workspace with no login. If local mode is disabled, protected workspace routes verify a Firebase Bearer token, including revocation, and scope records by uid. The current React API helper supplies no Firebase sign-in flow or token injection, so disabling local mode alone does not create a complete login experience. API keys authenticate ingestion separately.

Hosted design: what the repository configures

Docker uses two stages: Node builds static React files, then Python runs Flask under Gunicorn. render_wsgi.py builds the hosted application. DispatcherMiddleware mounts the task app at /source; Flask serves /api and frontend files, falling back to index.html for browser routes. Internal delivery uses Flask's test client with normal ingestion validation, without a separate network hop.

The configured Gunicorn process has one worker and four threads. One worker avoids multiplying the background sender/generator. The Render blueprint selects a Docker service with /api/health as its health check. Hosted code forces login-free local mode: public visitors can access records and workspace operations. The setup uses container-local SQLite files and provisions no persistent disk, so it does not guarantee durable data across container replacement. This explains repository configuration, not a verified live deployment or current hosting plan terms.

Troubleshooting by following the data path

No dashboard data? Check API /api/health, create a task, inspect task-service health, then the sender's queued_logs and data/*.stderr.log. **Growing queue?** Check endpoint/key and API availability; failed events remain queued. **No analysis?** Receive five logs with usable words, then Run analysis. **Unexpected counts?** Remember the 300-record display, 100-record clustering batch, retention, and generated scenarios.

Port already occupied? Inspect which process owns 5001, 8000, or 5173; the launcher skips occupied ports, and Vite requires its selected port. **AI says local?** Check backend key/provider settings. **Missing older logs?** Inspect retention and the selected database; old database files are not automatically the active store.

A short learning exercise and the remaining files

Create and complete one task, find its two event records, pause Explorer, run a small generated batch, run analysis, and ask "Investigate source:task-service". Explain each hop aloud. To extend the app: add an API route and validation, update the React API call/screen, and add a behavioral test for the new capability.